Auditable IoT Energy Data Pipeline

Delivery accounting, exact finalization, and independent verification

Project report 13 November 2026

Submitted by
Nishant Harkut
Enrollment number: 2023IMG-040

Instructor
Amandeep Kaur

Submission repository
`https://github.com/nishantharkut-research/IoT-Energy-Data-Pipeline`

Abstract

Smart-meter reports depend on a delivery and processing history that can include retries, delayed readings and malformed messages. This project builds a reproducible pipeline that preserves Kafka deliveries in Bronze, maintains a watermark-bounded Silver candidate, and reconstructs exact Gold records from Bronze after replay-completion checks. Independently implemented Hadoop Streaming programs reproduce source records and aggregates from canonical NDJSON. Finalization requires delivery conservation, complete partition termination, exact comparison agreement and bound result artifacts.

The evaluation uses public Hong Kong University of Science and Technology (HKUST) telemetry and controlled delivery fixtures. Fifteen runs are complete: nine correctness, recovery and watermark fixtures, with three clean repetitions each at 100K and 1M source records. All declared source and aggregate comparisons agree. A restrictive watermark leaves 143 records in Silver while Gold and Hadoop retain 144. Interrupted and uninterrupted fixture executions reproduce four semantic artifacts. Median verification costs are 13.88 minutes at 100K and 165.30 minutes at 1M. Consumption comparisons retain explicit coverage limits and unknown total exposure. A read-only dashboard presents the results and their provenance. The proposed Indian utility application is a review of metering-provider outputs; field effectiveness remains future work.

Evaluation boundary	Historical replay, one designated physical machine, up to 1M selected source records.
Submission update	Report and portable viewer use saved results. No new distributed workloads were executed for this submission.
Evidence available	Versioned result summaries, original scope and plan, source-audit summary, source code and tests.

Contents

1 Problem, objective and contribution **3**

2 Dataset and source preparation **4**

3 Canonical identity, time and numeric representation **5**

4 System architecture and technology choices **6**

5 Delivery processing and finalization **7**

6 Experiment design and controlled-fault results **8**

7 Independent Hadoop verification **9**

8 Watermark behaviour and consumption interpretation **10**

9 Recovery and reproducibility of the finalized answer **10**

10 Scale, storage and SQL measurements **11**

11 Dashboard, evidence access and demonstration **12**

12 Limitations, contribution and conclusion **12**

A Dashboard walkthrough **14**

Reading the results. Source availability, delivery conservation, computation agreement and consumption usability are reported separately. Agreement on selected canonical records does not establish that all original measurements were available. Screenshots show the actual read-only dashboard using committed summaries. They do not represent a new live experiment.

1 Problem, objective and contribution

A meter-data platform must distinguish the physical reading, its source row, each attempt to deliver that row, and the record eventually used in analysis. Confusing these identities can cause errors. A retry can create two deliveries of one source event. Two workbook rows can have the same timestamp. A valid record can arrive after the operational stream has released its deduplication state. A message can also fail the declared contract while remaining important evidence of what was delivered.

The project asks a practical question: *what evidence supports the completeness and correctness of a finalized result when the operational delivery path contains controlled faults?* Completing a Spark job answers only part of that question. The implementation therefore records deliveries, classifies their outcomes, reconstructs a finalized source result, and compares that result with a separately implemented computation.

1.1 Indian smart-meter data-review use case

The intended user is a distribution-company (DISCOM) analyst reviewing meter-data outputs supplied through an advanced metering infrastructure service provider (AMISP). A head-end system (HES) collects readings, while meter-data management (MDM) systems apply validation, estimation and editing (VEE). The review question is whether a reported result is adequately supported by the received data and its processing history.

A 30 October 2026 CSIS survey of TNPDCL and Tata Power-operated Odisha utilities reports communication failures and data-integration limitations [10]. These two environments motivate the use case; they do not establish a national failure rate. A CAG report published in 2025 also documents historical communication and missing reporting problems in a Tripura smart-grid project during 2018–2021 [11]. The distinction matters: installation counts alone do not establish that the data needed for analysis are available.

MSEDCL's 2022 procurement document already requires original received raw data, VEE data and change trails [7]. Commercial systems also provide quality and timeliness monitoring [8]. The proposed role is an inspectable review alongside this established infrastructure. The prototype is not a replacement for HES, MDM or VEE.

Table 1: How the Indian review use case connects to the implemented evaluation.

Analyst's question	Implemented mechanism	Available evidence
Was source data unavailable?	Profile published rows and retain quality exclusions.	The HKUST resampled grid has 19.04M missing numeric cells.
Did delivery behaviour alter the operational view?	Preserve Bronze, classify dispositions, rebuild exact Gold.	Combined fixture: 432 deliveries; restrictive Silver: 143 versus 144 in Gold.
Does a second implementation agree?	Compare independent Hadoop records and aggregates with Spark Gold.	All 15 saved runs agree on the declared exact fields.
Is a reported zero sufficiently supported?	Compare only matching consumption intervals and retain unknown exposure.	Two incomparable periods prevent a total energy-loss estimate.

The connection is about data accountability rather than a claim of measured Indian billing savings. The pipeline can reconstruct a valid reading that reached Bronze but was omitted by bounded Silver. It cannot recover a physical reading that never arrived. An Indian deployment would require authorized utility inputs, adapters for the actual HES/MDM exports, verified units and time conventions, agreed review rules, and field evaluation.

The engineering contribution is the reproducible integration of delivery accounting, exact source reconstruction, independent computation, recovery checks and explicit comparison coverage. Quality checking is established practice [9]; this project evaluates the integration with public HKUST telemetry and controlled delivery faults.

1.2 Questions evaluated

1. Can every received data delivery be assigned a recorded valid, duplicate, late or quarantine disposition?
2. Does finalized Gold agree with an independent implementation on every declared exact field?
3. Does interrupting and resuming the fixture change its semantic outputs?
4. Can a restrictive watermark change Silver while exact Gold remains unchanged?
5. What verification, storage and SQL costs were recorded for the completed 100K and 1M workloads?

The completed evaluation addresses these questions through controlled fixtures and selected real-data workloads. Its scope ends at exact processing of the received canonical records and the recorded verification, storage and query costs.

2 Dataset and source preparation

2.1 Public telemetry and metadata

The source is the HKUST campus smart-meter collection deposited on Dryad under DOI `10.5061/dryad.k3j9kd5h6`, with a companion data descriptor [1, 2]. The project registered the deposited version-5 files. Its counts describe those files, rather than substituting headline meter totals from the article. The package contains raw and publisher-resampled variants. Brick metadata provides semantic relationships and recorded units [3]. Raw and resampled records remain separate throughout preparation.

The published source audit records 1,394 raw workbooks and 1,329 resampled workbooks. All workbook meters in each variant join to Brick. The graph contains 1,432 meter references. Its additional references do not establish missing telemetry for a particular analysis window. The 65 raw workbooks absent from the resampled set have no physical data rows in the audit. This observation does not establish the authors' reason for excluding them.

Table 2: Published source-audit counts. Categories can overlap.

Observation	Raw	Resampled
Workbooks	1,394	1,329
Physical data rows	50,903,710	63,273,828
Finite numeric readings	50,903,710	44,238,638
Missing numeric cells	0	19,035,190
Negative readings	134,612	134,612
Ordered numeric decreases	50,675	67,878
Duplicate decoded timestamps	853,595	0
Out-of-order timestamps	147	0

2.2 What availability means

For the published resampled grid, the numeric-cell availability is

$$A_{\text{numeric}} = \frac{44{,}238{,}638}{63{,}273{,}828} \times 100\% = 69.916\%. \tag{1}$$

The denominator is the number of published rows. It is not an independently expected reading schedule. In particular, zero missing numeric cells in raw workbooks does not prove continuous temporal coverage. Negative values and ordered decreases are review observations, not confirmed meter failures or resets. Figure 4 presents these distinctions in the portable dashboard.

2.3 Registration and preservation

Registration records relative paths, byte sizes and SHA-256 hashes. Structural profiling examines workbook sheets and cells. The exhaustive telemetry audit records value, timestamp and ordering observations. Numeric canonical events and source-quality exclusions are retained separately. Source-quality exclusions are not Kafka quarantine: the former concern preparation, while the latter concerns delivered messages.

The prepared snapshot accounts for 63,273,828 source rows as 44,238,638 numeric records and 19,035,190 source-quality records. This accounting includes finite numeric values that may still have quality flags. It does not declare all numeric readings physically valid. Full audit JSON and row-level files were retained locally during the original work. This portable submission contains their published audit summary, not those large originals.

3 Canonical identity, time and numeric representation

3.1 A source row is different from a delivery

The source identifier is a SHA-256 digest of a structural locator:

$$\text{source_event_id} = H(v \,\|\, s \,\|\, p \,\|\, w \,\|\, r), \tag{2}$$

where v is dataset version, s is source variant, p is relative path, w is worksheet and r is physical row. The implementation serializes these fields with a declared separator and rejects ambiguous components. Identity does not depend on a value, timestamp alone, or a model-generated key. A repeated Kafka delivery therefore retains the same source identity but has its own delivery identity.

This choice matters when two physical rows share a timestamp. They remain distinct source events. Gold deduplicates repeated deliveries by exact source identity; it does not erase different physical rows merely because their measurement keys match. Consumption derivation applies a separate timestamp policy after source reconstruction.

Table 3: Selected canonical fields and their purpose.

Field group	Purpose
Dataset version, variant, path, sheet, row	Recover the original physical source location.
Source event identifier	Deduplicate repeated deliveries without merging distinct physical rows.
Original timestamp and numeric text	Retain stored representations for interpretation and exact conversion.
Event time and source timezone	Make the chosen timezone interpretation explicit.
Scaled value and decimal scale	Preserve numeric values with exact decimal arithmetic.
Unit and measurement kind	Restrict valid consumption derivation.
Brick identifiers and quality flags	Retain semantic relationships and observed limitations.

3.2 Exact values and an explicit timezone assumption

For a stored decimal x with scale d, the canonical value is the exact integer $z = x10^d$. The scale accompanies the value. The source reader retains stored cell text before conversion through a binary floating-point representation can change it. A bounded check found that meter D0779's stored value `26724.7`, at scale 15, requires integer `26724700000000000000`. It exceeds signed 64-bit range. The implemented storage contract uses signed values within a 38-digit decimal envelope, checked through the Spark representation path. This is a representation finding, not evidence of an incorrect physical reading.

All audited timestamps are naive. The executed snapshot interprets them in `Asia/Hong_Kong` under an explicit experimental assumption. The dataset authors did not confirm that convention. Original

timestamp text is preserved, and absolute UTC boundaries remain conditional on the assumption. The report does not reinterpret HKUST observations as Indian household measurements.

3.3 Preparation at bounded memory

The resumable preparation path creates sorted workbook units, writes checkpoints, and merges bounded groups into canonical NDJSON and the quality ledger. It subsequently writes Parquet and the snapshot manifest. The per-workbook checkpoints support preparation recovery without keeping the complete source in memory. They are intermediate work products, distinct from the immutable completed snapshot. The full source is prepared once; the scale runs select source-identifier-sorted prefixes from the numeric snapshot. Those prefixes are not guaranteed continuous meter-days and cannot support an outage-duration or time-of-use study.

4 System architecture and technology choices

Figure 1 shows the two computation paths. Source registration and canonicalization are shared. Kafka and Spark form the delivery path. Canonical NDJSON also enters HDFS for Hadoop Streaming. Both outputs are checked before finalization. The comparison is independent in implementation after canonicalization, not independent in physical measurement or source preparation.

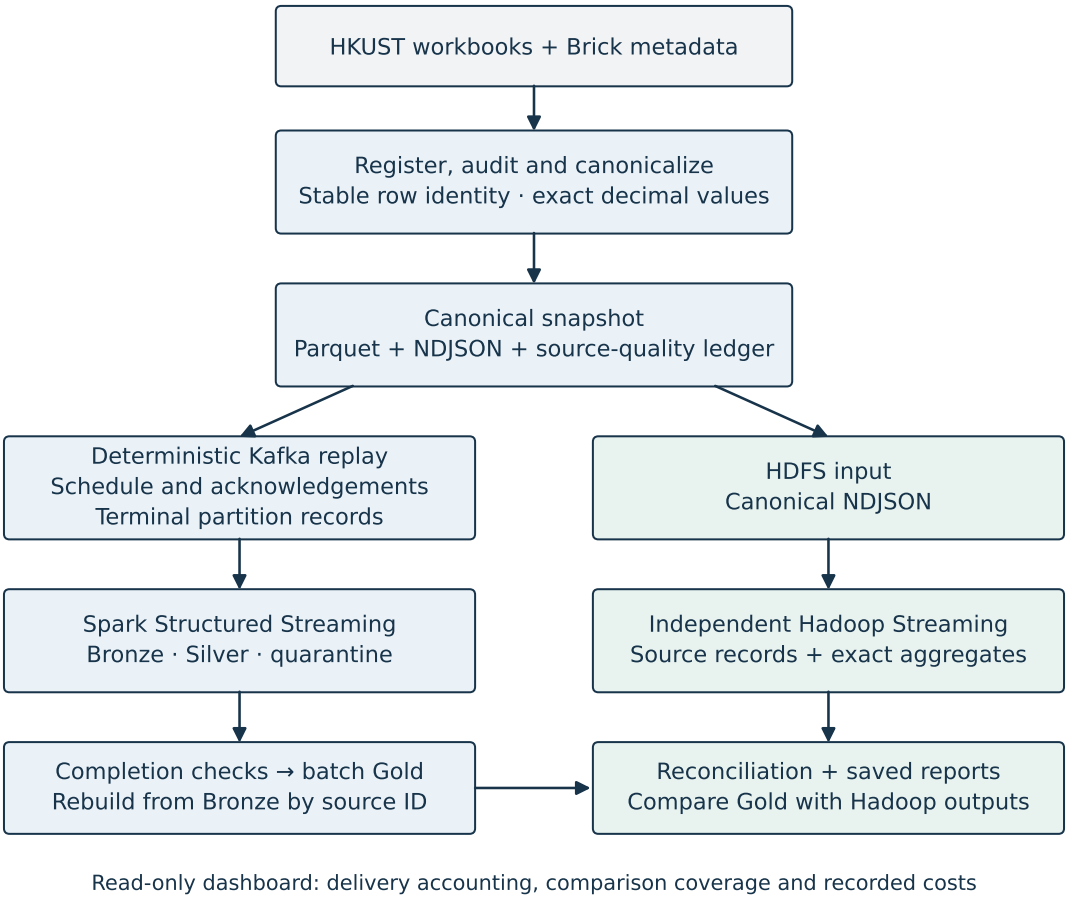


Figure 1: Processing path and independent reference computation share a declared canonical input.

Kafka supplies ordered partition logs, offsets and producer acknowledgements [5]. Spark Structured Streaming supplies event-time processing and bounded state [4]. Hadoop Streaming allows standalone

mapper and reducer programs to consume and emit records through standard input and output [6]. The independent programs do not import Spark transformations.

Parquet is used for columnar storage and Spark SQL. NDJSON is the explicit interchange format for independent processing and exact exported records. SQLite supplies disk-backed ordering and interval-processing support where needed. JSON manifests and reports supply bounded presentation inputs. These storage choices support historical replay and read-only reporting without introducing a separate application-serving database.

Docker Compose provides Kafka, Spark and Hadoop services for the original experiments. The portable dashboard uses React and TypeScript, Mantine interface components, Recharts plots and Lucide icons to present saved summaries. A Python export step preserves the exact evidence bytes; the browser checks their SHA-256 bindings. The original Streamlit sealed-run viewer remains available for inspecting original local artifacts. The portable viewer runs separately from the original experiment services.

5 Delivery processing and finalization

5.1 Replay records and terminal evidence

A deterministic replay schedule fixes the seed, source selections, partition assignments and injection types. Delivery records retain source identity and delivery identity. Producer receipts record acknowledgement information. Terminal partition records establish that the planned replay has ended for each partition. A quiet Kafka topic alone is insufficient proof of completion.

Bronze archives data deliveries with their transport evidence. Terminal controls are retained and checked separately. Parsing and contract validation then distinguish valid source events from malformed or contract-violating messages. Quarantine preserves rejected payloads with categorized reasons so that exclusion does not erase delivery evidence.

5.2 Operational Silver and finalized Gold

Silver is an operational candidate. Its event-time watermark bounds state and the period over which duplicate suppression can be maintained. Late valid deliveries are accounted for separately from invalid deliveries. Spark's documented watermark and deduplication semantics motivate this distinction [4]. Silver can therefore be useful while still being unsuitable as the sole source for an exact completed result.

Gold is a batch reconstruction from Bronze after completion checks. It parses valid deliveries, deduplicates exact source identities, and computes finalized source records and aggregates. This path retains received valid records that Silver omitted because of the operational watermark. It cannot recover a physical reading that was never supplied to the pipeline. Figure 2 shows the control and data paths.

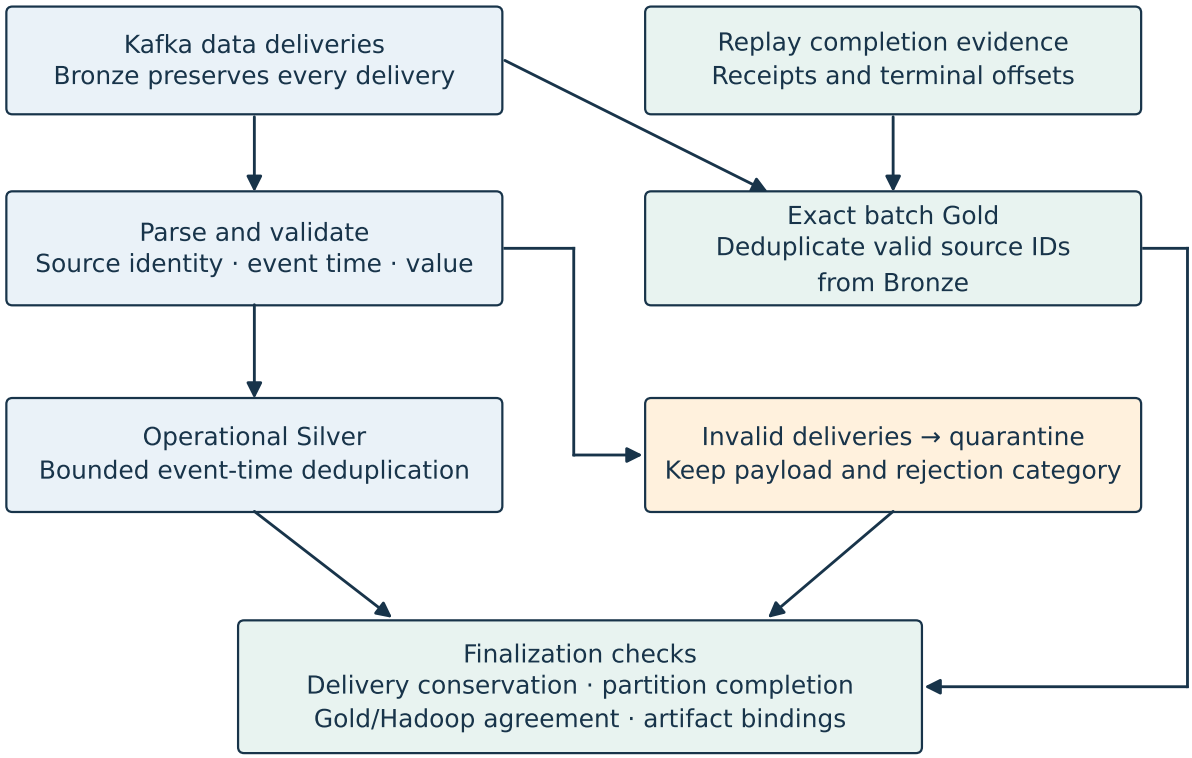


Figure 2: Delivery dispositions and exact finalization. Gold uses Bronze rather than inheriting Silver's omissions.

The layer audit checks two conservation equations. Let B be Bronze data deliveries, V parsed-valid deliveries, Q quarantine deliveries, S Silver records, D duplicate-valid dispositions and L late-valid dispositions. Under the recorded run definitions,

$$B = V + Q, \tag{3}$$

$$V = S + D + L. \tag{4}$$

The completion check separately binds acknowledged data deliveries to Bronze and verifies terminal partition coverage. The exact Gold count is a source-event count, not another delivery-disposition category.

5.3 Fail-closed publication of completed runs

Finalization requires the replay manifests, delivery totals, terminal offsets, layer dispositions and exact comparisons to reconcile. Decision, query and storage reports must bind to the corresponding finalized artifacts. A run manifest registers the artifacts and is sealed with a SHA-256 binding. The original sealed-run dashboard checks both this manifest and its registered files and rejects incomplete, altered or unreconciled runs. These checks establish byte bindings and declared processing consistency, not adversarial authenticity or physical accuracy.

6 Experiment design and controlled-fault results

The final matrix contains five correctness schedules, two recovery executions, two watermark configurations and six real-data scale executions. Each correctness schedule uses a 144-source-event fixture. The clean scale workloads use 100K or 1M selected canonical numeric records, with three repetitions at each scale. The originally planned 5M repetitions are explicitly excluded for resource and time constraints.

The declared services are Kafka 3.7.1, Spark 3.5.1 and Hadoop 3.4.3. Correctness and recovery fixtures use three Kafka partitions, at most 60 offsets per trigger, and a 24-hour watermark. Both watermark fixtures use one partition and one offset per trigger, with retaining and restrictive thresholds of 24 hours and one minute, respectively. Scale runs use three partitions and 10,000 offsets per trigger. Original machine metadata is hash-bound in the execution plan, but CPU and memory specifications are absent from the portable closure. The reported timings should therefore be interpreted as measurements from that original host, not directly transferable hardware benchmarks.

Table 4: Fixture layer counts from the saved closure report. Terminal controls are separate.

Fixture	Bronze	Silver	Dup.	Late	Quar.	Gold
clean	144	144	0	0	0	144
duplicate	288	144	144	0	0	144
delayed	144	144	0	0	0	144
malformed	288	144	0	0	144	144
combined	432	144	144	0	144	144
recovery-uninterrupted	144	144	0	0	0	144
recovery-interrupted_resumed	144	144	0	0	0	144
watermark-retaining	144	144	0	0	0	144
watermark-restrictive	144	143	0	1	0	144

The duplicate schedule repeats unchanged valid deliveries. The malformed schedule adds rejected messages alongside valid originals. The combined schedule includes both effects. Consequently, these schedules test delivery preservation and disposition under declared faults. They do not simulate arbitrary physical-meter corruption or guarantee a nonzero consumption error.

In the combined fixture, 432 Bronze deliveries split into 288 parsed-valid and 144 quarantined deliveries. The valid deliveries split into 144 Silver records and 144 duplicate dispositions. Gold contains the same 144 source records as the clean execution. The five correctness schedules have identical hashes for each of the four corresponding semantic outputs: Gold source records, Gold aggregates, Hadoop source records and Hadoop aggregates. This is stronger than merely reporting equal row totals.

The source and aggregate error lists are empty for all 15 completed runs. These results support the implemented fault-accounting and reconstruction behaviour under the selected schedules. They do not estimate the frequency of such faults in HKUST infrastructure or Indian AMI deployments.

7 Independent Hadoop verification

The Hadoop path reads canonical NDJSON rather than Spark's Gold result. Its standalone source-record program emits the declared exact representation, while the mapper/reducer path produces grouped counts, exact scaled-value sums, timestamp bounds and quality counts. Python integer arithmetic supports exact checksum accumulation within the declared contract. Group dimensions include the meter, unit, measurement kind and decimal scale, avoiding addition of incompatible representations.

Reconciliation sorts and compares exported source records by their declared identities and fields. It also compares aggregate records field by field. The source comparison covers 23 exact fields, including source location, original text, event time, scale, value, units, quality flags and Brick relationships. The aggregate comparison covers 10 fields. This makes the comparison contract inspectable rather than reducing it to an unqualified "passed" label.

Checksums of cumulative readings in aggregate records are useful exact processing checks. They are not energy-consumption totals. Consumption is computed separately from valid differences between supported successive readings. This prevents interpreting a large sum of meter-register values as electricity delivered during a period.

All 15 saved comparisons have matching source/aggregate counts and empty error lists. The largest checked source population is 1M selected canonical records. Independence ends at the shared canonical input boundary: both computations can agree on an incorrectly normalized input. The design therefore retains source-audit evidence and the assumptions used during preparation instead of treating cross-engine agreement as physical truth.

8 Watermark behaviour and consumption interpretation

The retaining watermark fixture preserves 144 source records in Silver. The restrictive configuration preserves 143, with the remaining valid record classified as late. Both executions produce identical finalized Gold and Hadoop source records. Thus the observed difference is an operational candidate omission, not a failure of exact Gold reconstruction.

Consumption is derived only for records whose declared semantics support cumulative kWh. For two eligible readings, interval energy is

$$\Delta E_{m,i} = E_m(t_i) - E_m(t_{i-1}). \tag{5}$$

The derivation requires a valid prior baseline and unambiguous timestamps. Negative differences and unresolved resets are flagged. The conservative duplicate-timestamp policy also excludes affected derivations and the first subsequent interval requiring a new baseline.

A numeric comparison requires the same meter, start time and end time in both series. Let $\mathcal{C}$ denote that comparable-support set. The recorded discrepancy is

$$D_{\mathcal{C}} = \sum_{(m,a,b)\in\mathcal{C}} \left| \Delta E^{\text{Silver}}_{m,a,b} - \Delta E^{\text{Gold}}_{m,a,b} \right|. \tag{6}$$

It is a subtotal over $\mathcal{C}$, not an error estimate for intervals outside it.

The restrictive execution has 140 candidate intervals and 138 verified intervals. There are 138 comparable periods and two incomparable periods. Candidate and verified flagged-record counts are three and six, respectively. The numeric discrepancy over comparable periods is zero. The two incomparable periods cannot be assigned a numeric error from these reports. The report therefore does not call them a 2 kWh loss or claim that verification saved that energy.

Scenario rates of 0.5, 1.0 and 2.0 currency units per kWh are declared hypothetical rates. They are neither HKUST tariffs nor Indian billing rates. A zero subtotal on comparable intervals yields a zero scenario subtotal there, while total energy error and residual financial exposure remain unknown. The project demonstrates preservation, reconstruction and careful interpretation of coverage; it does not demonstrate actual financial savings.

9 Recovery and reproducibility of the finalized answer

The recovery fixture compares uninterrupted execution with an interrupted-and-resumed execution. For each semantic artifact, its full SHA-256 digest is compared across the two executions. The four pairs agree. Transport offsets, timing files and physical Parquet layouts are not required to match, because they describe execution details rather than the declared semantic output.

The scale executions also required recovery. Saved receipts document resumed stages and configuration bindings. All three 1M runs finalized, but their complete workflow durations are unavailable. The report preserves that distinction. A recovered result can still support the recorded exact comparison and the measured component costs without supplying an uninterrupted end-to-end latency measurement.

The current submission did not rerun the pipeline. Its portable loader checks the bytes of committed summaries against an evidence index, verifies their original plan/scope bindings, checks the 15 run identities, and recomputes delivery-conservation equalities. These are new checks of the reporting package.

Original runtime verification is recorded at 2 November 2026 in the closure report. Reproducing the original per-artifact checks requires the original sealed directories, which are not included in a small Git checkout.

This separation avoids silently treating a screenshot as evidence that the original large files were freshly examined. The original strict loader remains accessible for users who retain those files. The portable mode instead makes the saved results usable without an attached SSD or an operating Kafka/Spark/Hadoop deployment.

10 Scale, storage and SQL measurements

10.1 Verification cost

Verification time includes Gold finalization, layer auditing, Hadoop computation and reconciliation. It excludes the streaming stage and subsequent storage/query benchmarking. Verification storage covers the declared finalized verification artifacts, not every checkpoint, log or service volume on disk. Table 5 reports the three repetitions at each scale. The 1M median is about 11.91 times the 100K median for

Table 5: Recorded verification costs. Time is minutes; storage is decimal MB.

Events	Minimum	Median	Maximum	Median storage
100,000	13.63	13.88	14.10	252.78
1,000,000	163.42	165.30	165.97	2522.71

a tenfold event-count increase. This is a descriptive ratio on one machine, not a claim of asymptotic complexity or production throughput. The absolute cost is relevant: full independent recomputation is substantially more expensive than showing a job-completion status. The current project measures that cost rather than claiming it negligible.

10.2 Storage formats

Table 6 compares the finalized NDJSON export with Zstandard Parquet outputs. Ratios are stored bytes divided by NDJSON bytes. The same ratio is recorded in all three repetitions at each scale. Unpartitioned compressed Parquet is smaller in these workloads. Partitioning adds file and metadata overhead; no benchmark here establishes that partitioning improves query speed.

Table 6: Recorded storage sizes and ratios for each scale.

Events	NDJSON (MB)	Parquet/NDJSON	Partitioned/NDJSON
100,000	112.04	0.0615	0.3038
1,000,000	1120.65	0.0601	0.1654

10.3 Query workloads and timing limits

Five read-only SQL workloads cover meter, building, time window, quality and exposure. Each runs three timing repetitions and records a complete ordered-result digest. The query policy uses cached Gold input and consumes projected JSON results, including serialization, transfer and hashing. These are complete cached-result workloads, not isolated SQL-operator latencies or cold-storage read measurements.

The 100K runs use the original settings. Smaller query partitions were used at 1M. Their timings therefore cannot be treated as a configuration-controlled scaling benchmark. A zero-row quality query in a selected numeric prefix also does not contradict full-source quality findings. Different populations and definitions

require separate denominators. Per-run minimum, median and maximum timings and result digests remain available in the committed results instead of being replaced by a single favourable run.

11 Dashboard, evidence access and demonstration

11.1 Nine views organised around review tasks

The entry view combines a selected-run pipeline with an experiment register. An analyst can filter and search the register, inspect a run, export its saved summary, or download the original complete result JSON. Source audit, delivery inspector, fault comparison, verification, watermark review, recovery, storage/SQL and India use case provide the remaining views. The interface uses plain section headings, compact metric rows, tables and a consistent light theme. Units remain visible, and wide tables scroll locally. Figures 3–11 show actual browser captures with navigation and page context intact. They are grouped in Appendix A to keep each capture large enough to read. Each landscape capture preserves a 1280 by 900 desktop viewport. Longer content continues below it; full-page PNGs and a run-inspector dialog capture are retained in `docs/submission/figures/dashboard/`.

The controls operate on saved evidence. Switching source variants changes the displayed population; selecting a run changes its counts and contract; the watermark switch exposes the retaining/restrictive distinction. CSV and JSON exports contain the selected saved records. No control initiates a new experiment or invents a live meter feed.

The portable mode loads only bounded files already in the repository. An evidence index binds the closure report, original scope, original experiment plan and published source-audit summary. Missing or altered inputs prevent the view from displaying affirmative results. The loader also checks count conservation and recorded comparison agreement. Its result is explicitly a validated saved-summary presentation. It is not a replacement for per-artifact verification of the original sealed experiment directories.

The separate sealed-run mode retains the original strict artifact checks. A user who supplies a valid local run directory can inspect the original six sealed views. Opening an absent or incomplete run produces an error. The default portable mode avoids requiring a multi-gigabyte evidence tree merely to review the submitted results.

11.2 A short demonstration

A six-minute demonstration can begin with the overview, then show source quality and explain the published-row denominator. The combined-fault view demonstrates the delivery equations. Independent verification shows the declared comparison fields and experiment families. The watermark view explains the 143/144 distinction and unknown total exposure. Recovery and storage/SQL show reproducibility and cost. The India view closes with the intended analyst, the supporting sources and the limits of the evaluation. No live 1M replay is needed.

A fresh checkout can launch the React viewer with Python 3.11 or newer for its standard-library evidence exporter and Node.js 24 for the locked frontend dependencies. Run `npm ci` and `npm run dev` from `dashboard/web/`. The separate strict viewer uses the documented Python/Streamlit environment. The README links this report, its LaTeX source, screenshots and machine-readable results. The original dataset is obtained from Dryad separately. It is unnecessary for reviewing the portable submission, but required for reproducing source preparation.

12 Limitations, contribution and conclusion

The implemented contribution is an evidence-preserving pipeline that combines deterministic delivery schedules, an operational stream, exact batch reconstruction, independent computation, recovery checks and a read-only evidence presentation. Its value is demonstrated by concrete distinctions: deliveries

differ from source events, Silver differs from exact Gold, and computation agreement differs from source adequacy.

Several limits determine how the results can be used. The workloads are historical replay on one physical machine. Fault schedules are controlled rather than estimated from an Indian utility network. The scale selections are source-identifier prefixes rather than continuous meter-days. Canonicalization is shared by both engines. Absolute timestamps depend on an unconfirmed timezone assumption. Physical measurement accuracy, tariff correctness, live-system reliability and actual financial savings were not evaluated. The final scale evaluation ends at 1M selected source records.

The zero comparable-energy subtotal is a valid scoped result. It also reflects the chosen experiment and comparison definition: duplicate deliveries retain unchanged values, malformed deliveries accompany valid originals, and unmatched interval supports are not priced. A cumulative-register series can lose an intermediate observation without changing the energy difference between unchanged endpoints. The project therefore reports the observable accounting and coverage effects with their comparison coverage.

All 15 saved runs pass their declared source-record and aggregate comparisons. Exact Gold retains the valid source event omitted by restrictive Silver, and the interrupted fixture reproduces four semantic artifacts. These findings support the implemented recovery and independent-verification design through 1M selected source records. The proposed utility-review workflow requires further evaluation on authorized utility data before operational adoption.

Further work could evaluate realistic communication-loss histories, continuous-window consumption effects or cheaper selective verification. Such extensions would require new baselines and experiments. They are outside this submitted scope.

References

[1] M. Li, Z. Wang, Y. Qu, K. M. Chui and M. Leung-Shea, "A multi-year campus-level smart meter database," *Scientific Data*, vol. 11, article 1284, 2024. doi:10.1038/s41597-024-04106-1.

[2] HKUST smart-meter dataset, Dryad, deposited version 5, 1 August 2024. doi:10.5061/dryad.k3j9kd5h6. Registration and package details: repository `docs/dataset-audit.md`.

[3] Brick Schema, "Brick ontology and documentation." `https://brickschema.org/`. Accessed 13 November 2026.

[4] Apache Spark, "Structured Streaming programming guide," 3.5-series documentation. `https://spark.apache.org/docs/3.5.6/structured-streaming-programming-guide.html`. Consulted for watermark and deduplication semantics. Experiment image: Spark 3.5.1.

[5] Apache Kafka, "Design," version 3.7 documentation. `https://kafka.apache.org/37/design/design/`. Accessed 13 November 2026.

[6] Apache Hadoop, "Hadoop Streaming," version 3.4.3 documentation. `https://hadoop.apache.org/docs/r3.4.3/hadoop-streaming/HadoopStreaming.html`. Accessed 13 November 2026.

[7] Maharashtra State Electricity Distribution Co. Ltd., AMISP tender MMD/T-NSC-06/0922, 19 September 2022, section 2.4.4, PDF p. 185. Tender document.

[8] Oracle Utilities Analytics, "Performance," Meter Data Management metric reference. Quality, timeliness and estimation documentation. Accessed 13 November 2026.

[9] E. Garrison and J. New, "Quality control methods for advanced metering infrastructure data," *Smart Cities*, vol. 4, no. 1, pp. 195–203, 2021. doi:10.3390/smartcities4010012.

[10] S. Kumar and A. Subramaniam, "From Meters to Machine Learning: Building AI-Ready Utilities in India," CSIS, 30 October 2026. Original utility survey and analysis.

[11] Comptroller and Auditor General of India, Report No. 2 of 2025, Tripura, year ended 31 March 2024, chapter IV, printed p. 144. Historical smart-grid project observations.

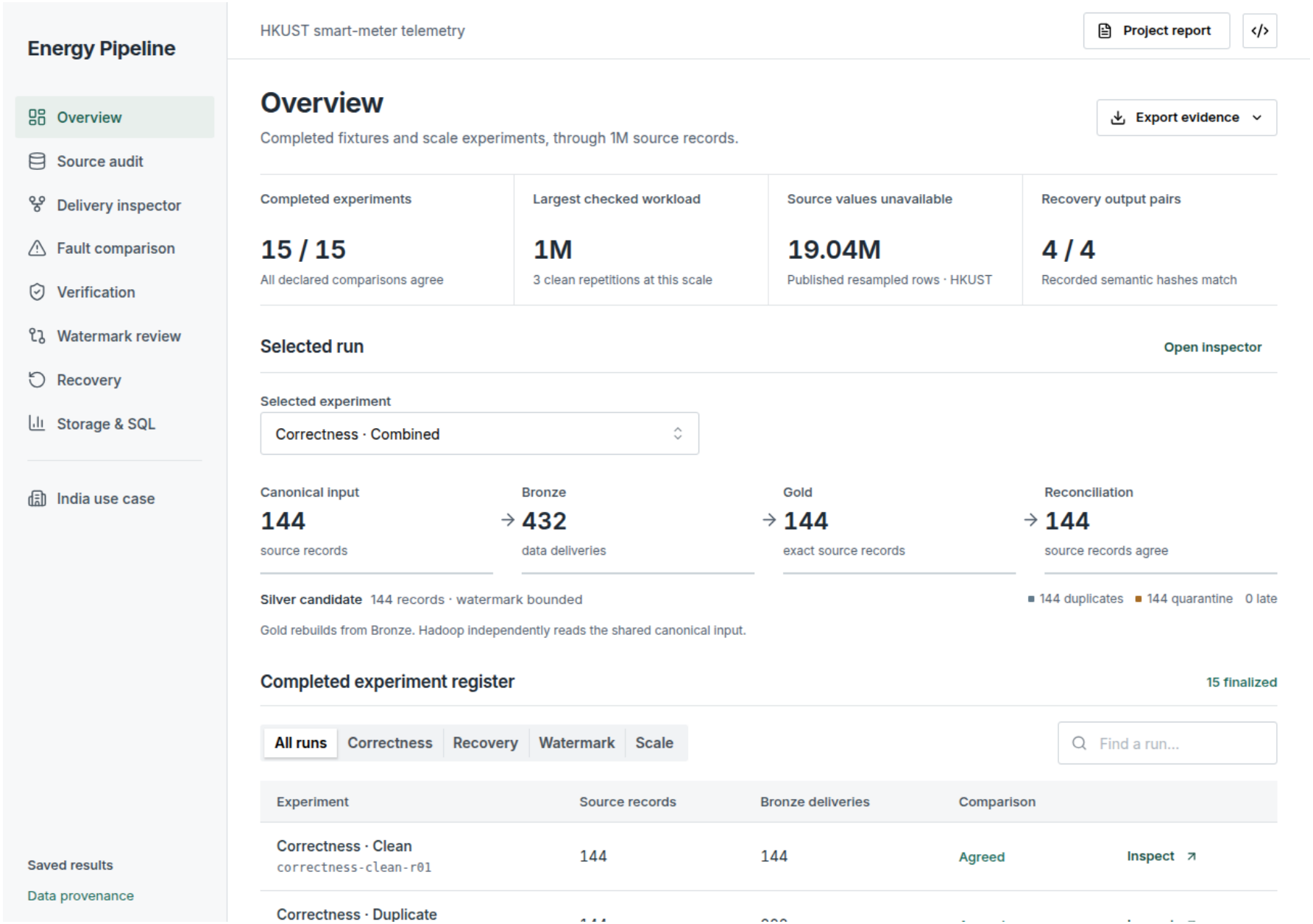


Figure 3: Select a recorded run, follow its delivery accounting, or search the completed experiment register below.

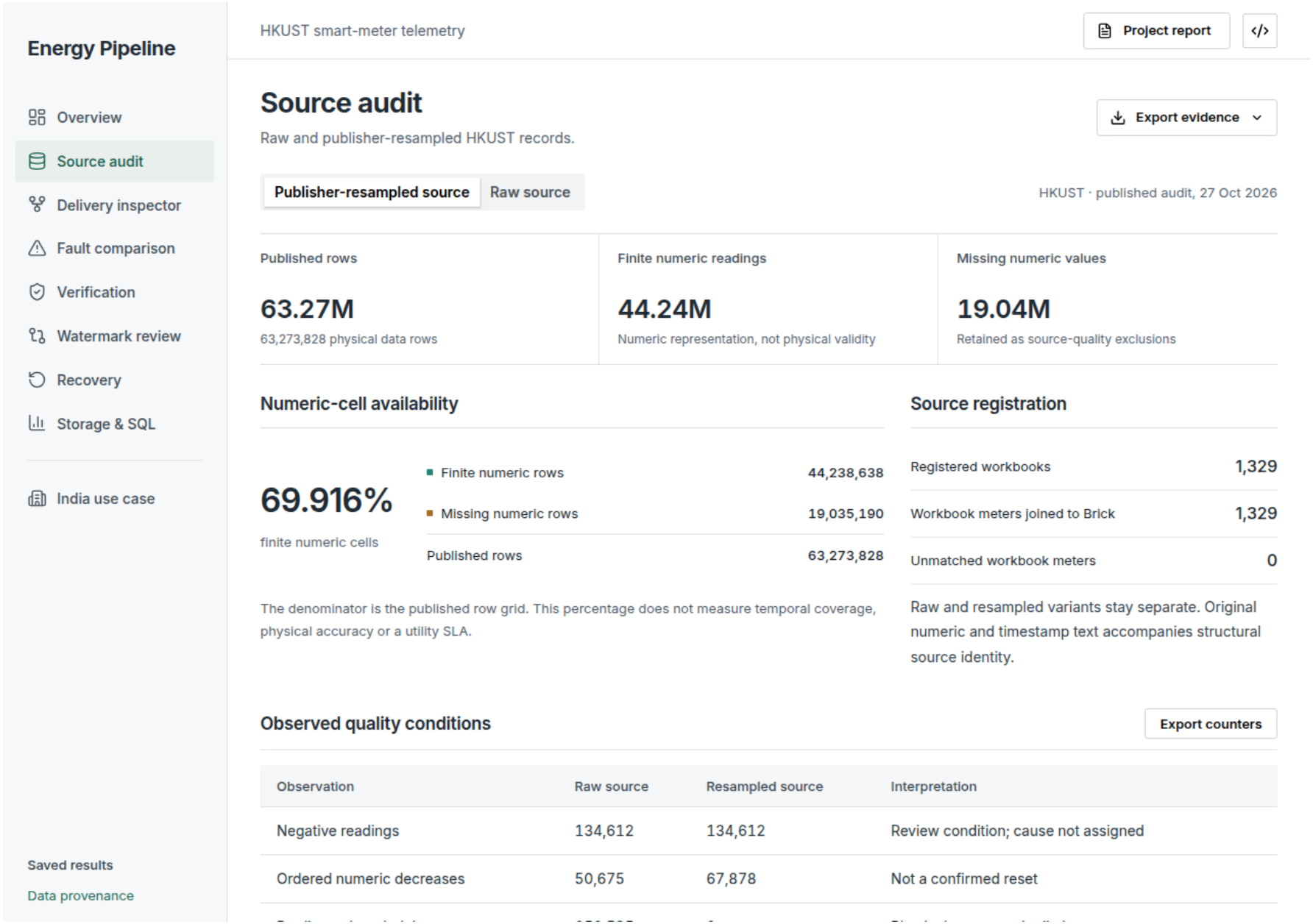


Figure 4: Switch between raw and publisher-resampled populations. Numeric-cell availability uses published rows as its denominator.

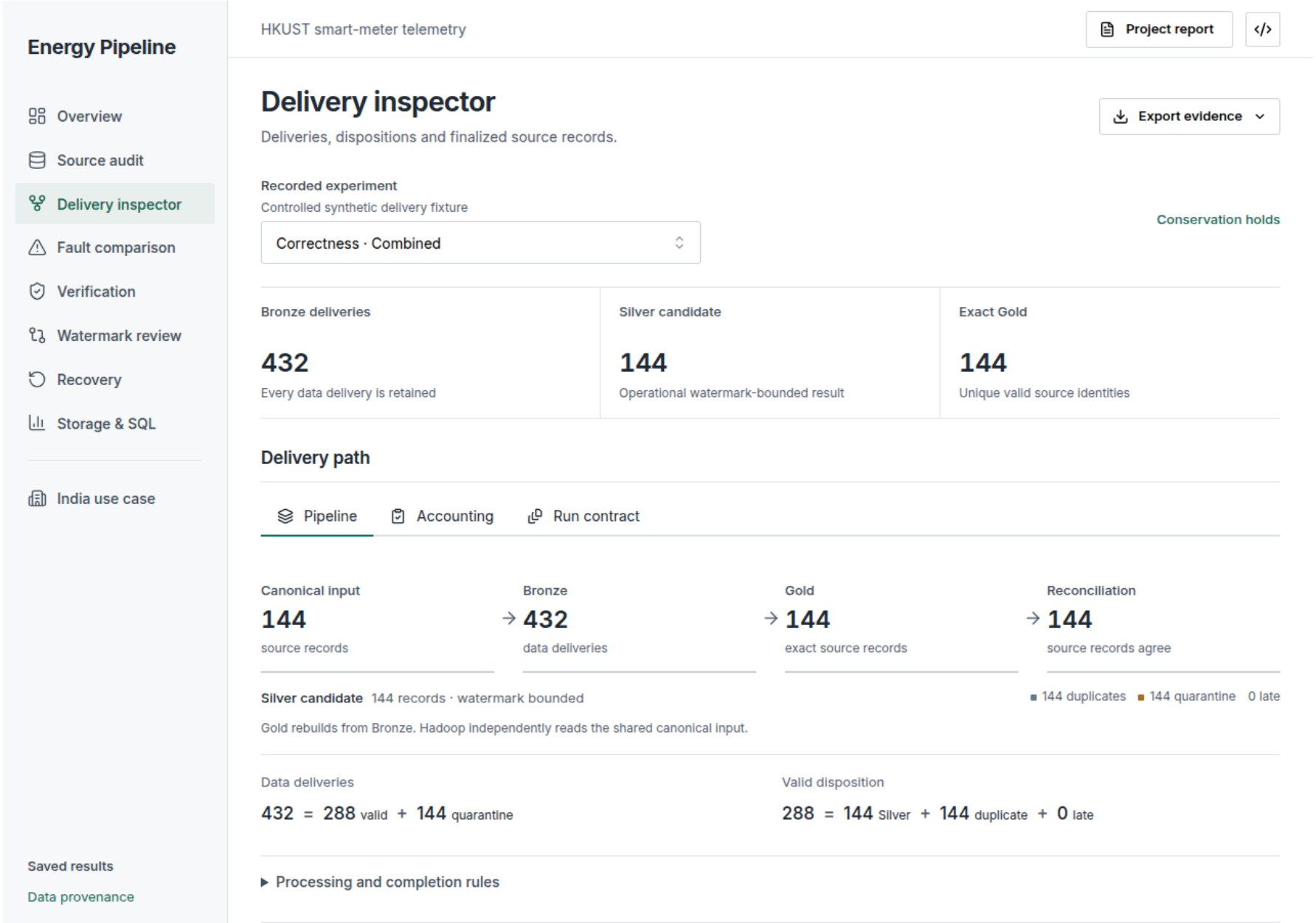


Figure 5: Inspect one run. Pipeline, accounting and run-contract tabs separate the processing path from its recorded configuration.

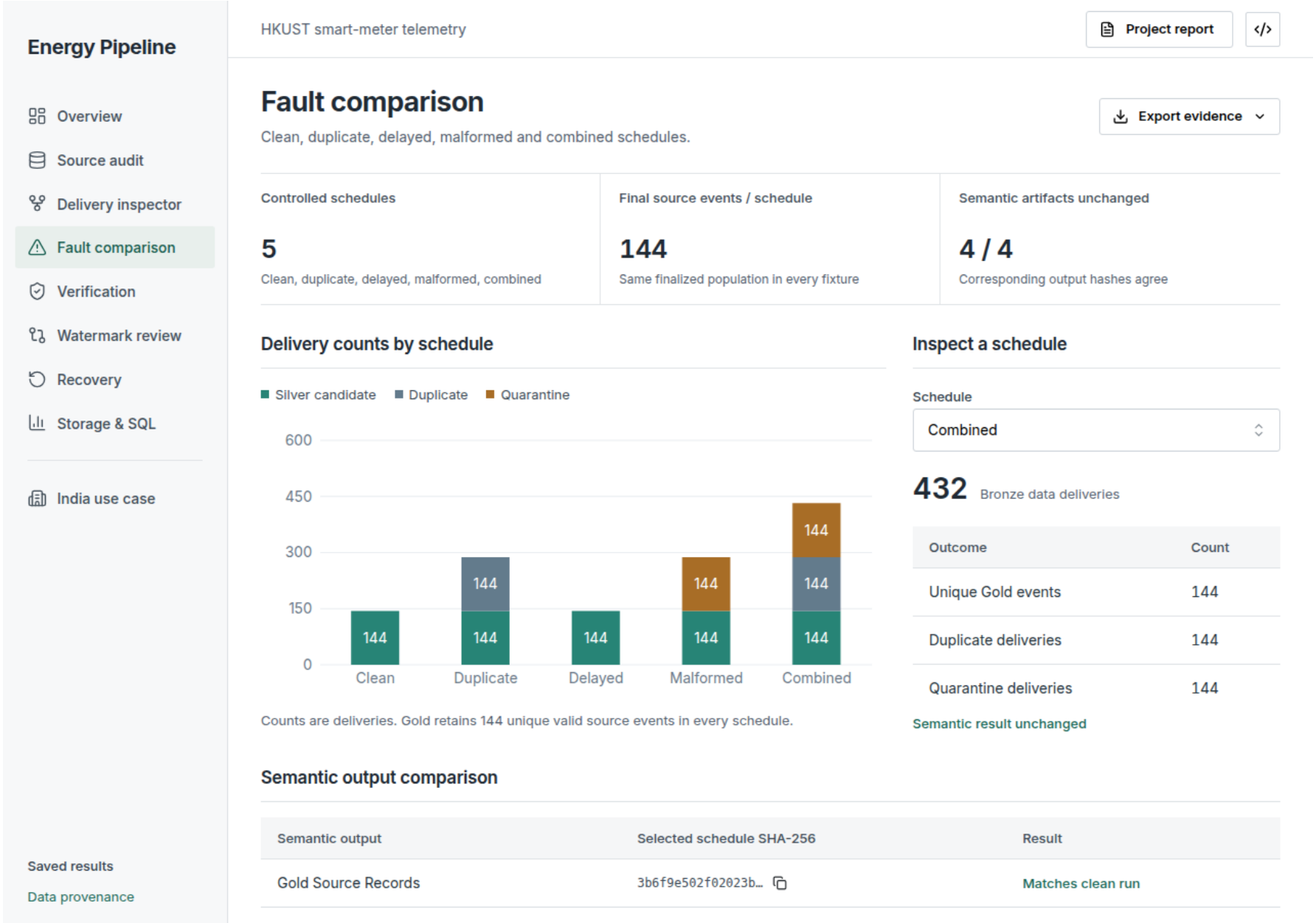


Figure 6: Compare five deterministic schedules. Duplicate and quarantined deliveries change while the exact finalized source result is preserved.

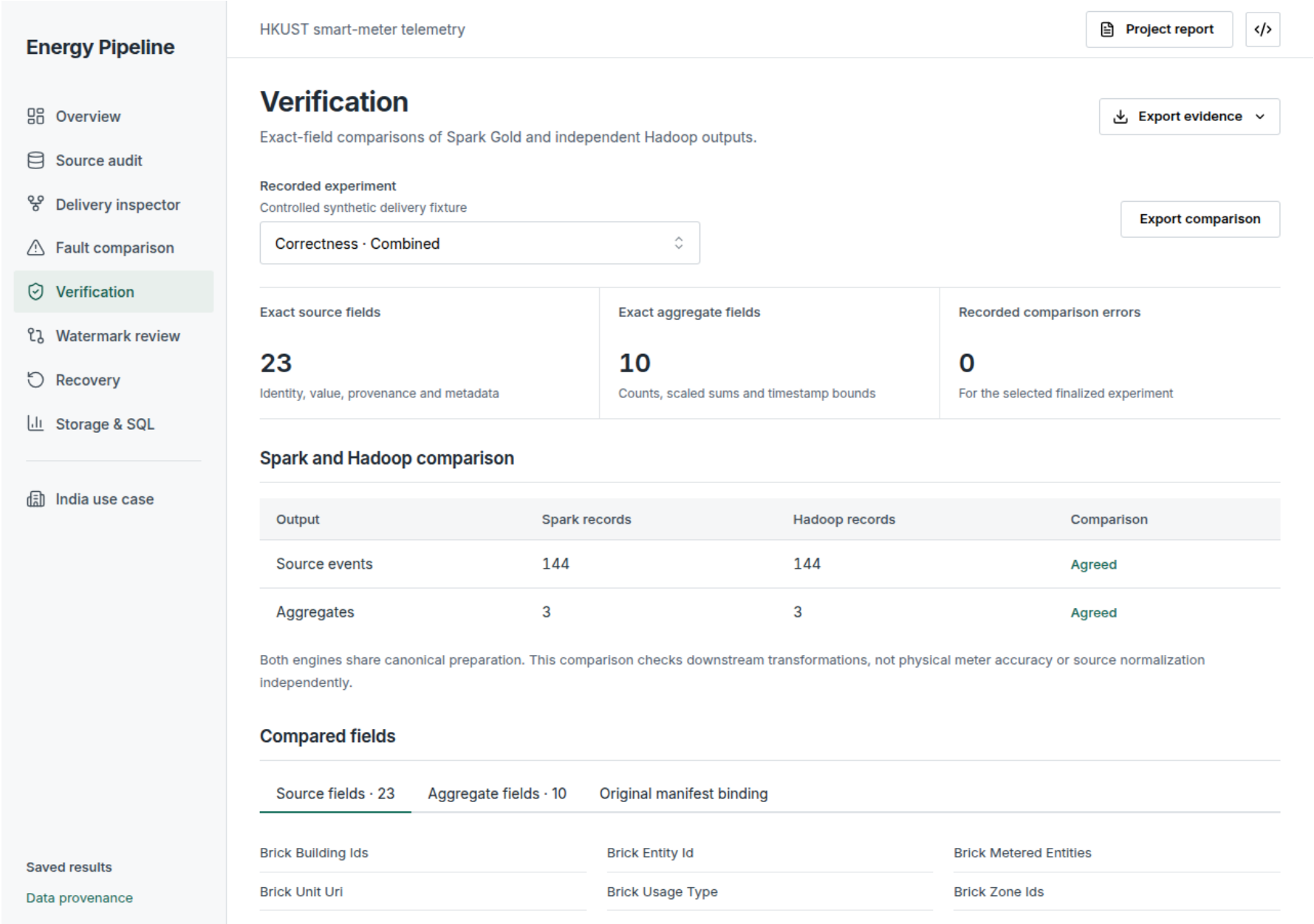


Figure 7: Review source and aggregate comparisons from independently implemented Spark and Hadoop transformations over shared canonical inputs.

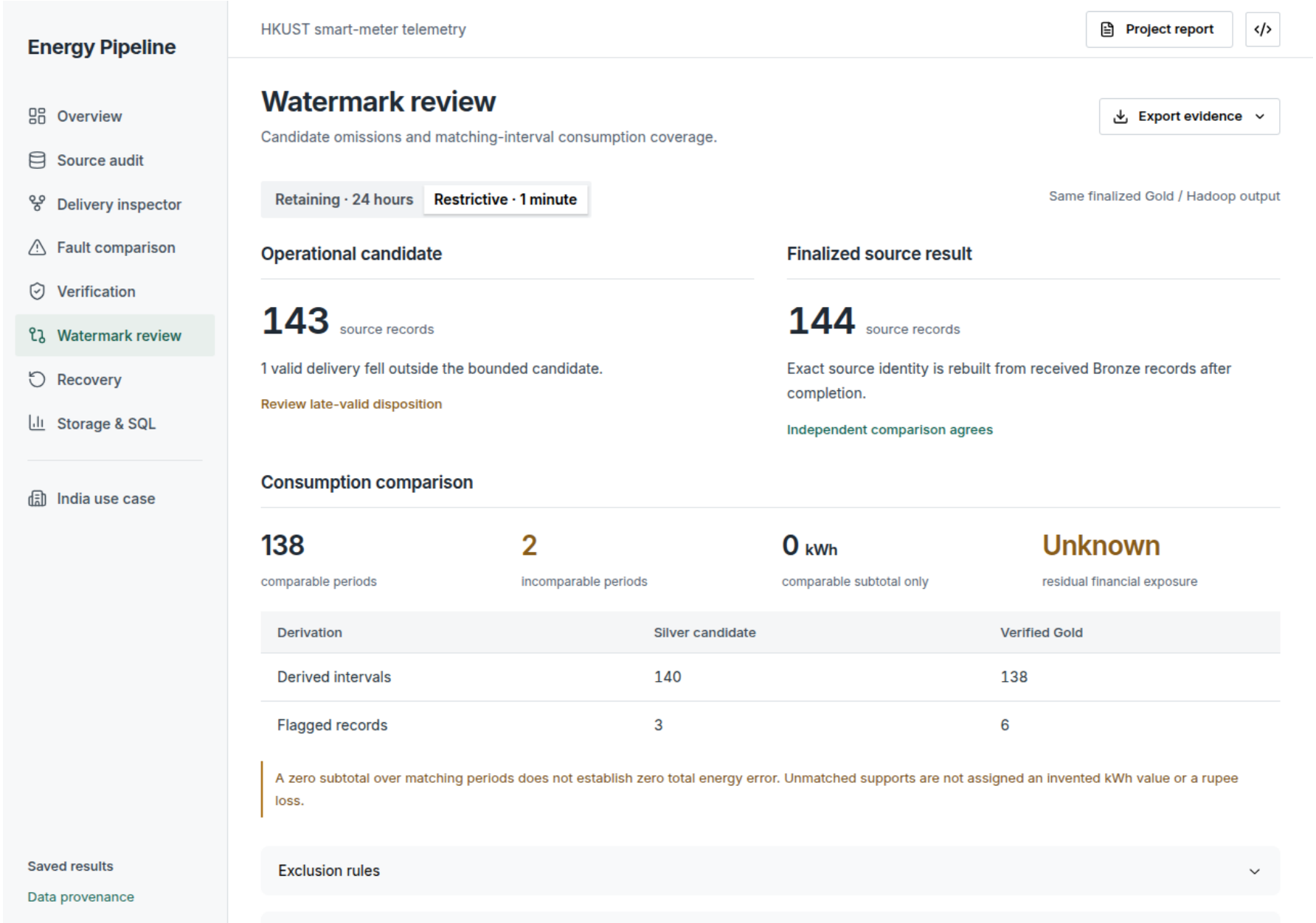


Figure 8: Switch watermark policies. The restrictive candidate omits one event; matching-interval energy and unknown total exposure remain distinct.

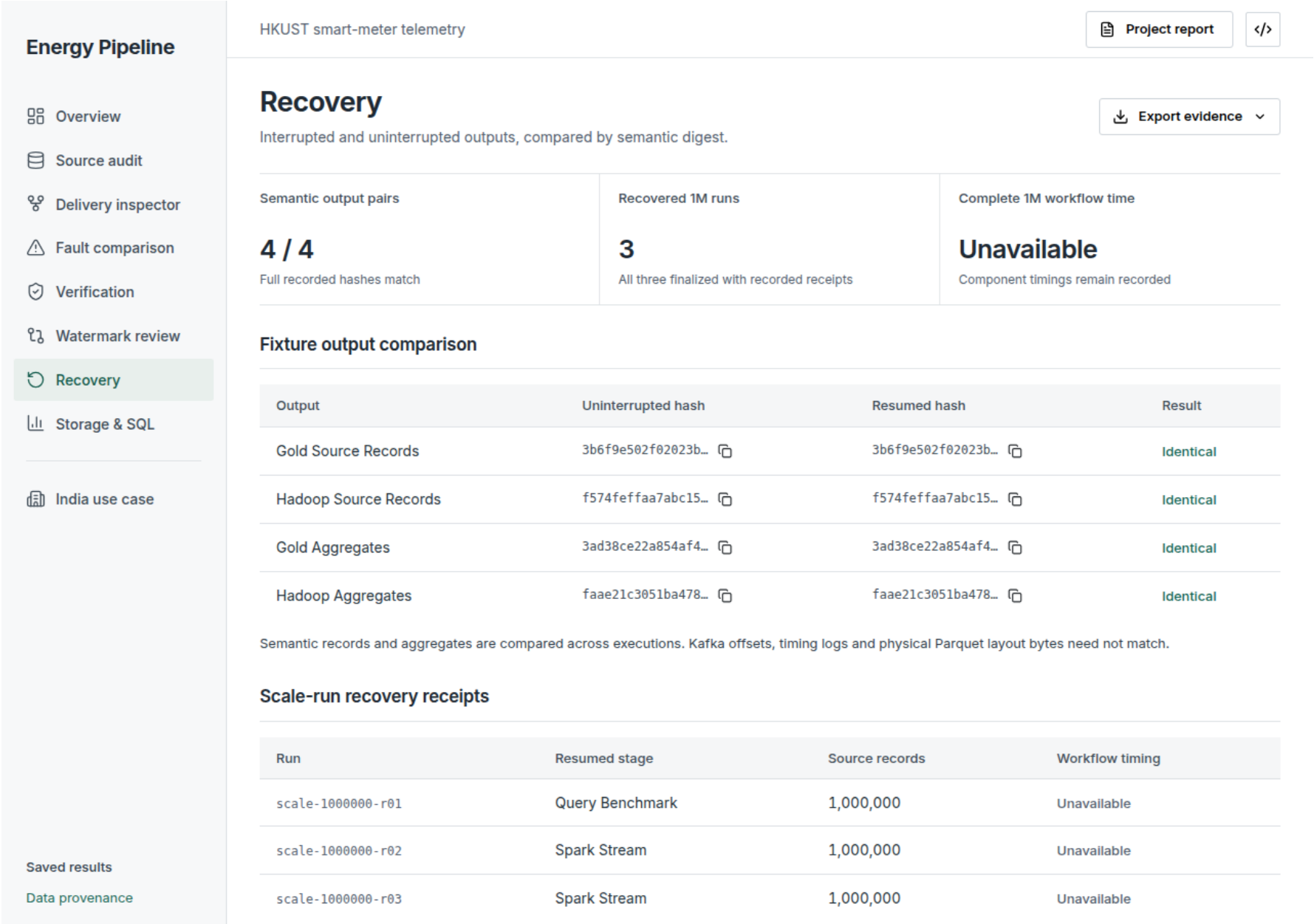


Figure 9: Inspect the four matching semantic output pairs and the separate timing limitations for recovered scale executions.

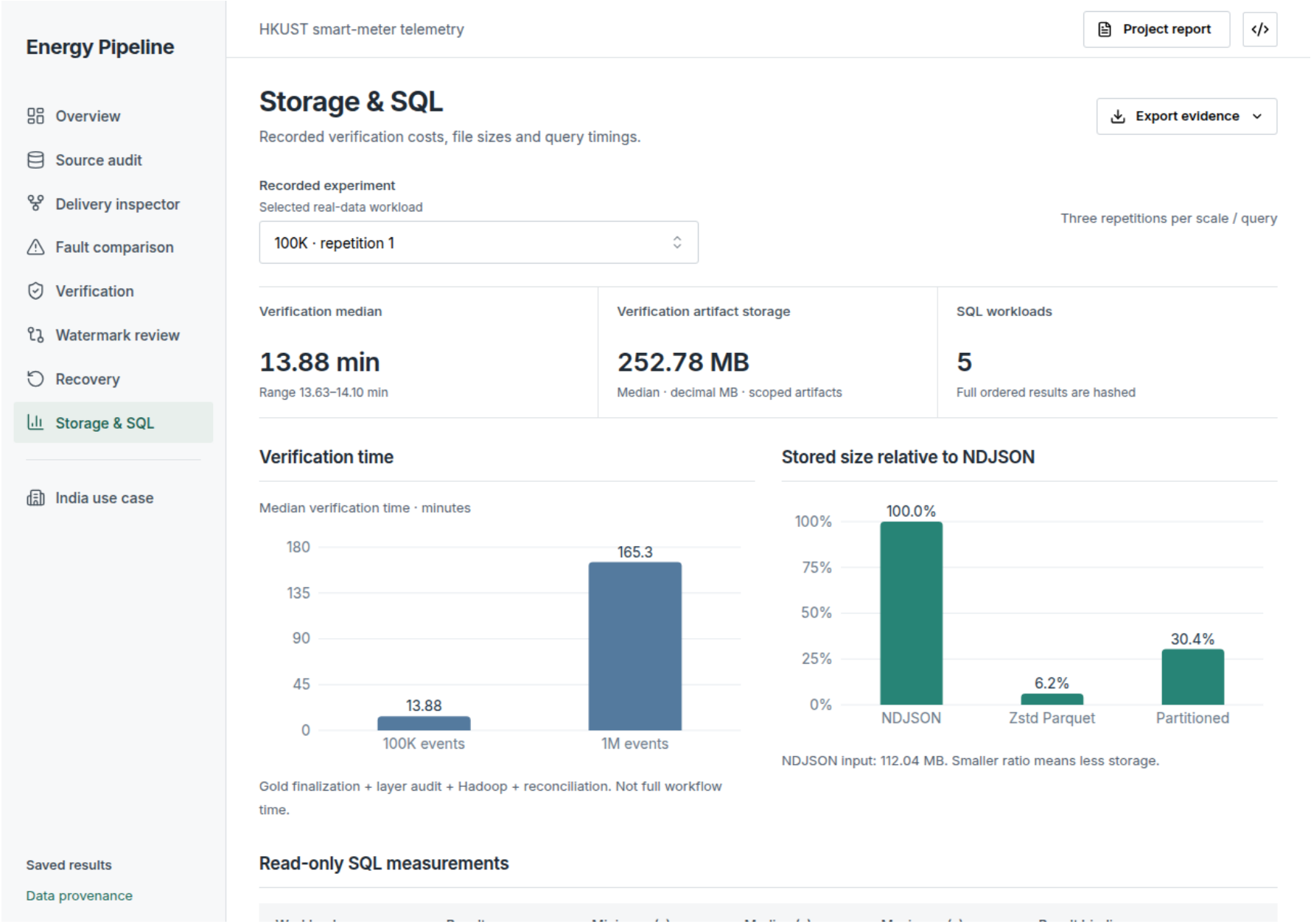


Figure 10: Choose a recorded scale repetition. Verification cost, storage ratios and ordered-result SQL bindings have explicit measurement scopes.

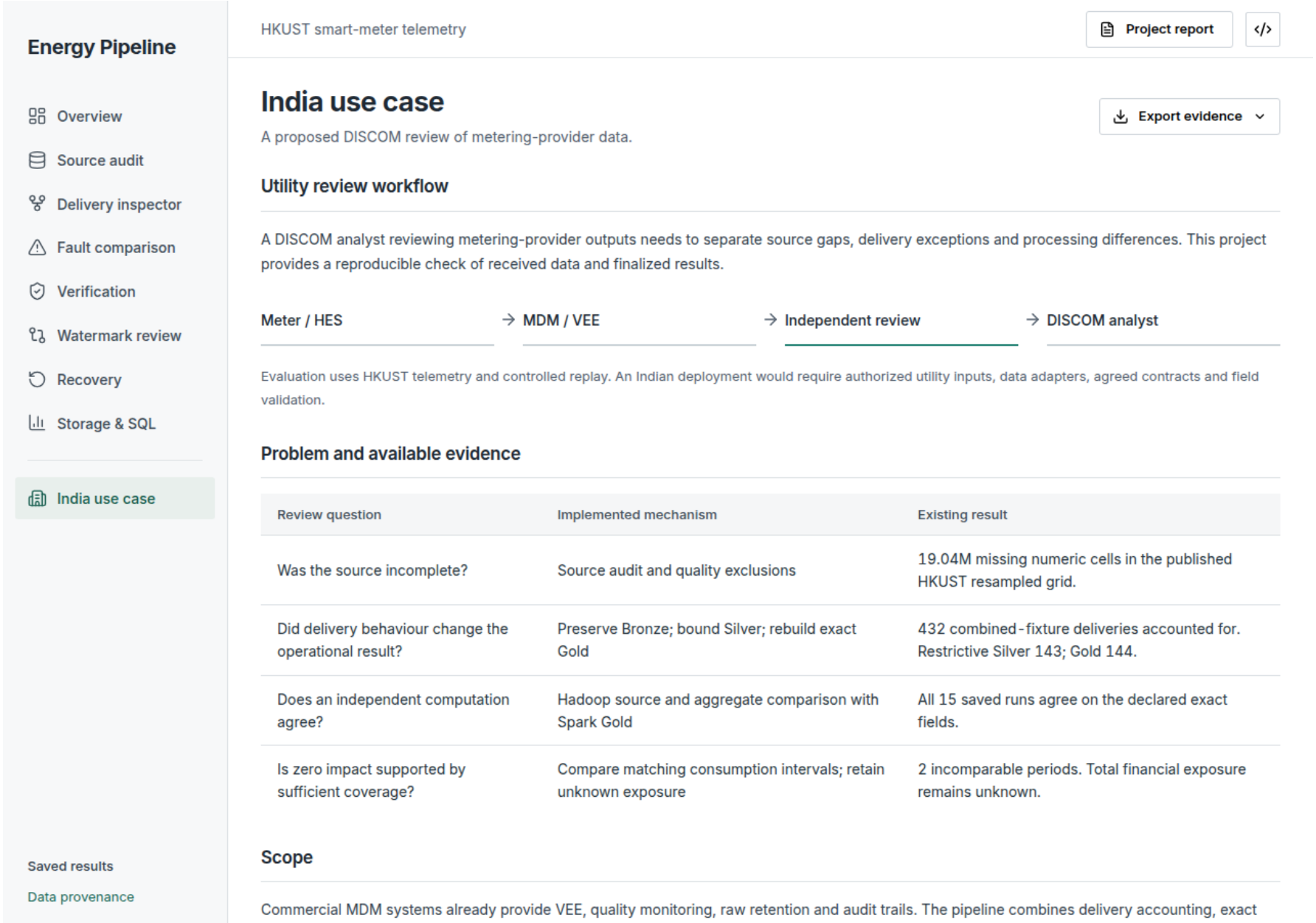


Figure 11: Locate the proposed independent review alongside an existing HES/MDM. The full view includes problem-to-evidence mappings and dated sources.